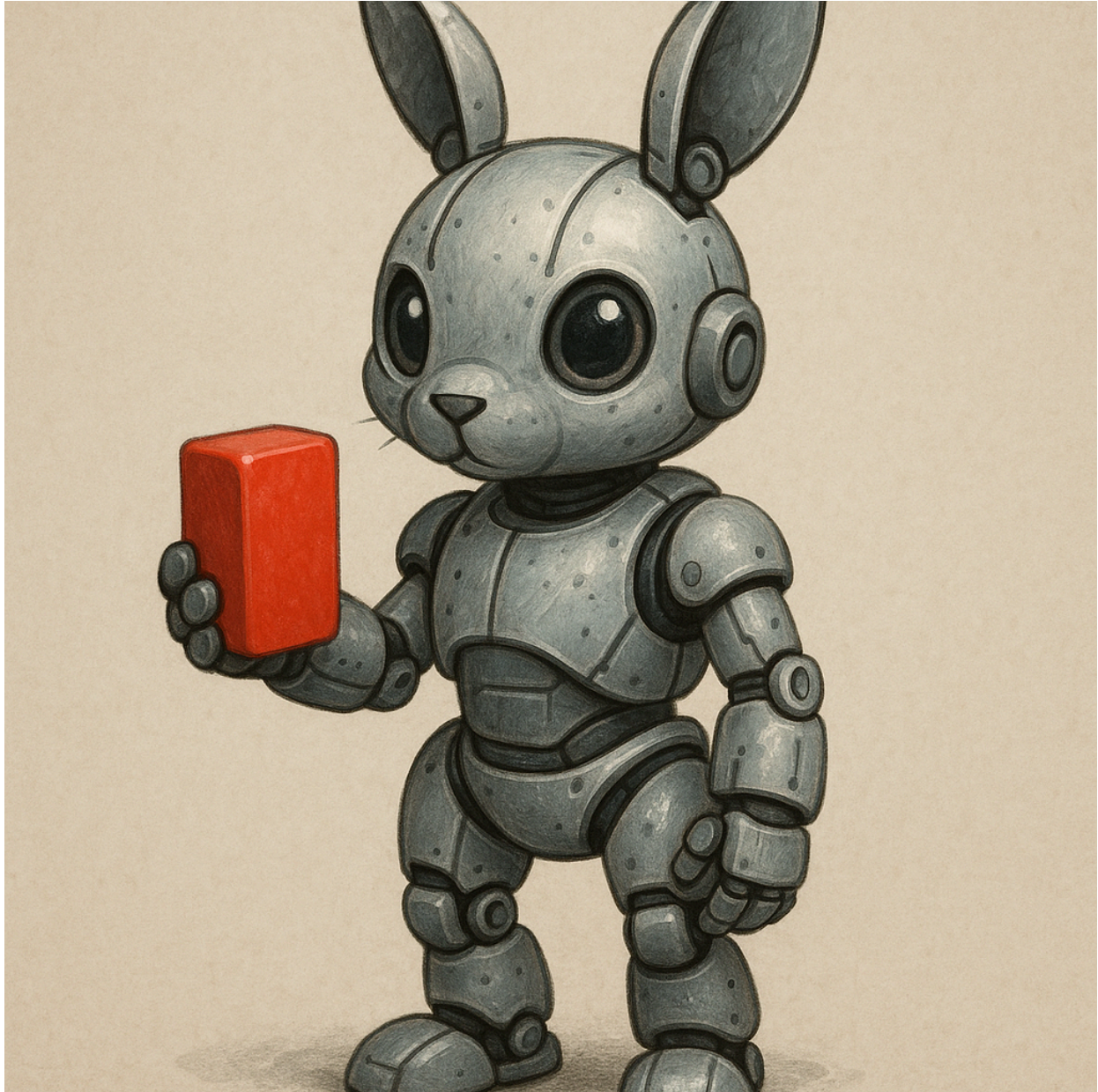


Warehouse Sorting Robot Arm (Lab-bot)

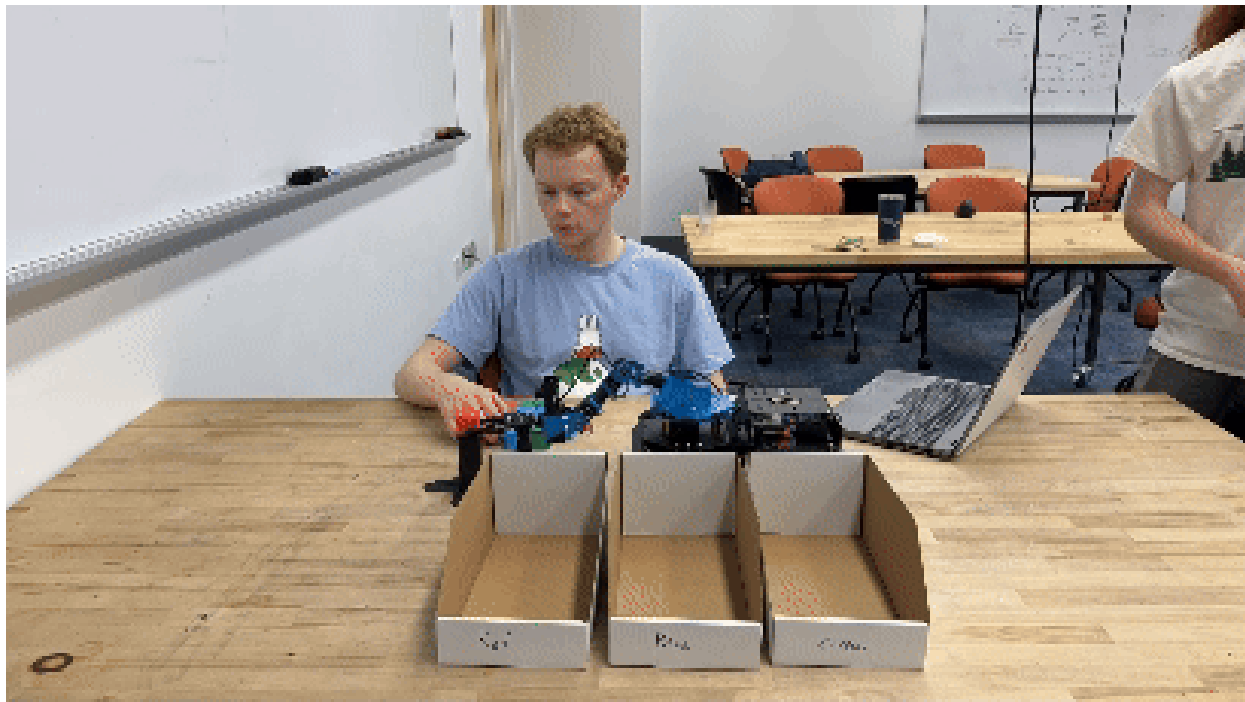
Rowan and Reuben



Introduction

Our final project was to design and implement a small-scale warehouse sorting robot using a 5-DOF robotic arm, vision-based detection, and pre-recorded movement scripts. The robot's task was to pick up colored blocks from a fixed location, use a camera to detect their color, and

sort them into the correct bin. This work draws inspiration from real-world automation in industrial and prototyping environments, and is motivated by the desire to streamline small-part sorting in cluttered workspaces.



Project Goals:

- **MVP:** Pick and sort a single block based on color using pre-set movements and a simple vision system.
- **Stretch Goals:** Handle stacks of blocks and dynamically adjust Z-height. Possibly operate with randomized positions.

"You start your day with a clean lab and all your parts are put away neatly in bins. But as you work, things get cluttered. Lab-bot will run through the night to put all the parts back in their place"

Methodology and Implementation

System Overview:

Our system is composed of the following components:

- Hiwonder 5-DOF robot arm with gripper
- Gamepad interface for live control and recording
- OpenCV-based color detection
- Pre-computed kinematic scripts for block pickup/drop
- Vision-triggered script switching to determine where to drop a block

Software Architecture:

- The main program (`main.py`) monitors gamepad input and switches between movement scripts.
- Movement scripts are stored as CSV files (e.g., `pick\_block.csv`, `drop\_red.csv`, `drop\_blue.csv`) and contain lists of 6-joint angle positions and a dwell time for the movement.
- A `vision.py` script captures a single image, applies an ROI, and returns the dominant color using basic RGB analysis.
- When the robot detects a block, it executes the `pick\_block.csv`, uses `vision.py` to identify the block's color, and then runs the corresponding `drop\_<color>.csv` script.

Kinematics:

We implemented our own forward and inverse kinematics pipeline based on the Denavit-Hartenberg convention. Key modules include:

- `calc\_forward\_kinematics()` for live position tracking
- `calc\_inverse\_kinematics()` for converting task-space waypoints into joint angles using an analytical decoupled wrist solution.

Path Planning:

The kinematic architecture relies on a series of task-space way-points defined using XYZ coordinates and RPY Euler angles.

By chaining together the following 7 positions, we are able to achieve the sorting action.

```
home_pos = [0.22, 0, 0.1, 0, 1.57, 0]    #in front of the block tower
block_pick_up = [0.32, 0, 0.115, 0, 1.57, 0]  #at the block to be picked
up
block_lift_up = [0.32, 0, 0.2, 0, 1.57, 0]  #above the block tower to
clear the stack
block_drop_off_1 = [0.21, 0.27, 0.25, 0, 1.57, 1]  #Red bin drop off
block_drop_off_2 = [0, 0.3, 0.25, 0, 1.57, 1.57]  #Blue bin drop off
block_drop_off_3 = [-0.21, 0.27, 0.25, 0, 1.57, 1]  #Green bin drop off
drop_off_clear = [0.2, 0, 0.3, 0, 1.57, 0]    #Drop off clear height to
prevent collision with boxes
```

To go from task-space to joint-space, we use analytical IK.

```
def calc_inverse_kinematics(self, EE: EndEffector, soln=0):
```

```

"""
    Calculate inverse kinematics to determine the joint angles (theta1
through theta5)
    based on a desired end-effector position and orientation.

    Args:
        EE (EndEffector): Object containing desired x, y, z position and
roll (rotx),
                                pitch (roty), and yaw (rotz) orientation in
Euler angles.
        soln (int, optional): Optional parameter for handling multiple IK
solutions. Defaults to 0.

    This method updates self.theta with the joint angles and clips them
within
    specified joint limits.
"""

# Initialize joint angles
self.theta = [0, 0, 0, 0, 0]

# Step 1: Compute wrist position (position of joint before wrist)
R_0_to_5 = euler_to_rotm((EE.rotx, EE.roty, EE.rotz)) # Convert
desired orientation to rotation matrix
H_ee = np.eye(4)
H_ee[0:3, 0:3] = R_0_to_5
H_ee[0:3, 3] = [EE.x, EE.y, EE.z]

# Back out the wrist center from the end-effector position
wrist = np.array([[EE.x], [EE.y], [EE.z]]) - ((self.l5 + self.l4) *
(R_0_to_5 @ np.array([[0], [0], [1]])))

# Step 2: Convert wrist position to cylindrical coordinates
l1, l2, l3 = self.l1, self.l2, self.l3
r = np.sqrt(wrist[0]**2 + wrist[1]**2) # Radial distance in XY plane
z = wrist[2] - l1 # Height from base

# Step 3: Solve first three joint angles (shoulder, elbow)
self.theta[0] = np.arctan2(wrist[1], wrist[0])[0] # Base rotation
(theta1)

```

```

        self.theta[1], self.theta[2] = self.twoDOF_ik(r[0], z[0], l2, l3,
soln) # Shoulder and elbow angles
        self.theta[1] = -(np.pi/2 - self.theta[1]) # Convert to DH frame
convention
        self.theta[2] = -1 * self.theta[2] # Invert for proper elbow
direction

        # Step 4: Clip the joint angles to respect joint limits
        for i in range(len(self.theta)):
            self.theta[i] = np.clip(self.theta[i], self.theta_limits[i][0],
self.theta_limits[i][1])

        # Step 5: Solve wrist orientation (theta4 and theta5)
        # Rebuild forward kinematics up to joint 3
        self.DH[0] = [self.theta[0], self.l1, 0, np.pi/2]
        self.DH[1] = [self.theta[1] + np.pi/2, 0, self.l2, np.pi]
        self.DH[2] = [self.theta[2], 0, self.l3, np.pi]

        H_0_to_3 = np.eye(4)
        for i in range(3):
            T = dh_to_matrix(self.DH[i])
            H_0_to_3 = H_0_to_3 @ T

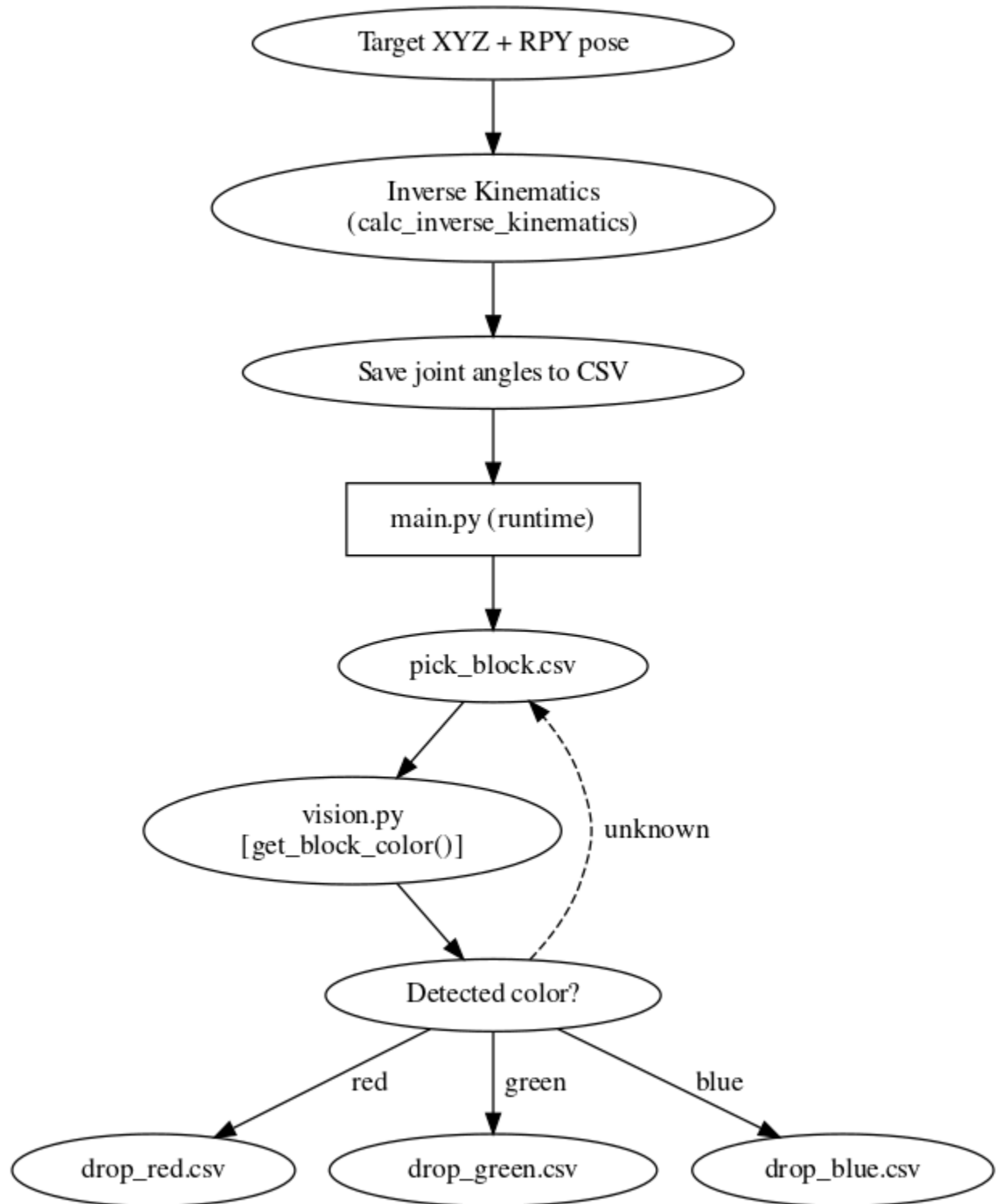
        R_0_to_3 = H_0_to_3[0:3, 0:3]
        R_3_to_5 = R_0_to_3.T @ R_0_to_5 # Relative rotation from joint 3 to
end-effector

        # Solve wrist pitch (theta4)
        if soln == 0:
            self.theta[3] = np.arctan2(R_3_to_5[1,2], R_3_to_5[0,2])
        else:
            self.theta[3] = np.arctan2(R_3_to_5[1,2], R_3_to_5[0,2]) #
Placeholder for alternate solutions

        # Solve wrist roll (theta5)
        self.theta[4] = np.arctan2(-R_3_to_5[2,0], -R_3_to_5[2,1])

```

Block Diagram:



Results

We were able to:

- Home and initialize the robot reliably
- Detect red, green, blue, and neutral (wood) blocks with consistent accuracy
- Use pre-recorded movement scripts to pick and drop blocks into bins with satisfactory reliability.

Result Documentation

Final Demo Video:

https://drive.google.com/file/d/1v_JsurPUu-JeB1aaHnAeeiVz9nF5S7-J/view?usp=sharing

Reflections and Conclusion

What worked well:

- The vision system was lightweight but effective for simple color classification
- Pre-recording movements reduced compute needs and made execution snappy
 - Using the viz tool for movement validation also helped to parallelize development.
 - For the MVP goal, there was no need to use a more complicated approach - our method worked well enough to meet the goals.
- Modular file structure made it easy to tweak behaviors without code changes

Main challenges:

- Calibration was critical: a few degrees error would cause dropped blocks
- Without Z-height sensing, stack handling required manual height tuning
- Threading and real-time coordination between vision and movement took debugging
- Consistent color detection with shadows and motion blur

What we'd improve:

- Add height sensing (using ArUco tags or stereo vision)
- Use HSV color space for more robust color detection in variable lighting
- Add logging and emergency stop for safer testing